

MINI-PROJECT SHORT TECHNICAL REPORT

Course: Cross-Platform Mobile App Development (VKU)

Mini-Project Title: Mini-Project 2: VKU Room Booking & Real-Time QR Check-In System

Team / Student Name: Nguyễn Thị Thương

Submission Date: 04/10/2026

1. GENERAL INFORMATION & DELIVERABLE LINKS

- **Team Members:**
- Nguyễn Thị Thương — Student ID: 23IT.B219 — Lớp SH: 23SE4 — Lớp HP: Phát triển ứng dụng di động đa nền tảng (4) — Role: Full-Stack Developer — Contribution: 100%
-  **Live Demo URL:** <https://vku-room-booking-17t.pages.dev/>
-  **GitHub Repository:** <https://github.com/thuong614oh65/vku-room-booking-expo>
-  **APK Download Link:** [app-release.apk \(78.8 MB\)](#)
-  **Video Demo (Optional):** N/A

2. FEATURE IMPLEMENTATION CHECKLIST

#	Required Feature	Status	Implementation Details & Acceptance Level
1	Responsive Mobile Viewport	✔ Complete	100% responsive across mobile viewports via React Native Web; dark navy header, bottom tab navigation.
2	Local Offline Persistence	✔ Complete	Uses Zustand + <code>@react-native-async-storage/async-storage</code> for local caching of bookings and user state.
3	Automatic Background Sync	✔ Complete	Auto-syncs booking data to Google Cloud Firestore upon network reconnection (<code>isSynced</code> flag queue).
4	Multi-Parameter Filter	✔ Complete	Real-time filtering by keyword, building (Khu A/B/C/V/Lib), capacity, equipment, and date/time slot.
5	VKU 2-Hour Slot Grid	✔ Complete	5 standard VKU time slots (07:30–19:30) with visual locking for occupied slots in <code>TimeSlotGrid.tsx</code> .
6	First-Committed-Wins Booking	✔ Complete	<code>atomicAddBooking()</code> in Zustand store prevents duplicate bookings via synchronous <code>checkSlotConflict()</code> .
7	Smart Conflict Resolution	✔ Complete	Modal recommends alternative rooms (± 10 seats), adjacent open slots, or priority waitlist entry.
8	Priority Waitlist	✔ Complete	Students queue for booked slots; push notification dispatched automatically when slot is released.
9	Digital Booking Pass & QR Code	✔ Complete	Electronic pass with Student ID, room code, slot time, UUID, and scannable QR matrix.
10	Physical Camera QR Scanner	✔ Complete	Hardware camera barcode scanner with flash toggle and instant CHECKED_IN status update.
11	Booking Lifecycle Management	✔ Complete	<code>MyBookingsScreen</code> with tabs (Upcoming / Checked-In / Cancelled) and one-tap cancellation.
12	15-Minute Reminder Notification	✔ Complete	Background scheduler sends push alert 15 minutes before check-in time.
13	Cloud Firestore 2-Way Sync	✔ Complete	Real-time bidirectional sync via Google Cloud Firestore REST API for multi-device consistency.
14	Native Android APK	✔ Complete	Production APK (78.8 MB) built via Expo EAS Build, installable on Android 10–15 devices.

#	Required Feature	Status	Implementation Details & Acceptance Level
15	PWA Standalone Mode	✅ Complete	Full PWA with 192/512px icons, Service Worker v15 (Cache-First), and install prompt.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE

- **Directory structure:** App root `App.tsx` → `src/screens/` (BrowseRooms, RoomDetails, BookingConfirmationPass, MyBookings, QRScanner, Profile) → `src/components/` (RoomCard, FilterBar, TimeSlotGrid, ConflictResolutionModal, InstallAppModal) → `src/hooks/useRoomsQuery.ts` (TanStack Query) → `src/store/useBookingStore.ts` (Zustand) → `src/services/` (queryClient, notificationService, realtimeBookingService) → `public/` (manifest.json, sw.js, icons).
- **State management flow:** Hybrid State Architecture: **TanStack Query** (`@tanstack/react-query`) handles server state and room caching (`useRoomsQuery`), while **Zustand store** (`useBookingStore.ts`) handles client state, session, and atomic booking transactions with `AsyncStorage` persistence and sub-millisecond `BroadcastChannel` synchronization.
- **Exception handling:** (1) Concurrent booking collisions handled by synchronous pre-flight check before state mutation. (2) Offline mode: bookings queued with `isSynced: false` and reconciled on reconnection. (3) Camera permission denied: QR scanner shows graceful fallback modal instead of crashing.

4. EMPIRICAL EVIDENCE & SCREENSHOTS

- Screenshot 1 — Browse Rooms Screen:* Shows dark navy header with 🟢 Online / ⚡ Real-time badges, building filter chips (Khu V, A, B, C, Lib), and room cards loaded via 60 FPS `FlatList`.
- Screenshot 2 — Room Details & Slot Grid:* Shows 7-day date selector and 5 VKU time slots; available slots highlighted in blue, booked slots locked in red with 🔒 icon.
- Screenshot 3 — Digital Booking Pass:* Shows student pass with Nguyễn Thị Thương / 23IT.B219 / 23SE4 / room V.A201 / slot time / UUID / QR matrix.
- Screenshot 4 — QR Scanner:* Shows hardware camera viewfinder with scan frame overlay, flash toggle button, and instant check-in result popup.

5. TECHNICAL CHALLENGES & RESOLUTIONS

- **Challenge 1 — Concurrent Booking Race Condition:** When two students book the same slot simultaneously, async writes can cause duplicates. Resolution: implemented synchronous `checkSlotConflict(roomId, date, slotId)` inside `atomicAddBooking()` so only the first commit is confirmed; the second triggers `ConflictResolutionModal` with alternative rooms and slots.
- **Challenge 2 — PWA Install Prompt Not Firing:** Chrome requires exact 192×192 and 512×512 icon dimensions in `manifest.json`. Initial mismatch suppressed `beforeinstallprompt`. Resolution: generated correct-size icons, upgraded Service Worker to v15 with `skipWaiting()` + `clients.claim()`, and built a 3-state button (📱 Cài Đặt App → 🚀 Vào App → 🟢 Đang Trong App) using `matchMedia('(display-mode: standalone)')` and `appinstalled` event listeners.